Java Recap

Variables

Java holds values in **named** storage locations, commonly called variables. There are many different types of variables to hold different things: int or Integer for integer values, double and float for floating-point numbers, Boolean for true or false, Character or String for character(s), and so on. You declare or instantiate a new variable by defining two things: (i) the type, and (ii) the name. Optionally, you can also instantiate a new variable with a specific value.

Example: `int my_number = 10;`

This declares a new variable of name my_*number* of type int. Furthermore, this variable is initialized with value 10.

Attention: depending on the variable, you may not assign values of a different type than the one used to declare it. For example, we cannot assign a floating-point number (such as 3.14) to our variable *my_number*, as this variable stores only integer values.

Conditional Statements (if)

The conditional statement, i.e. the if statement, allows a program to carry out **different actions depending on a certain condition**. Relational operators (<, <=, >, >=, == (equal), != (different)) are used to compare numbers and Strings.

Example: Print a specific message if my_number variable is equal to 10; print a different message otherwise.

```
if (my_number == 10) { // if condition is True, then enter if statement and print the message
	System.out.println("Specific Message");
}
else { // otherwise, we enter the else and print the “Different message”
	System.out.println("Different Message");
}
```

Attention: Do not use the == operator to compare Strings. Use the .equals method instead.

Finally, multiple conditions and if statements can be combined to evaluate complex decisions. In this case, we use conditional operators (&& - represent an *and* operator, || - represent an *or* operator) to combine multiple conditions.

Example: Check if a number is equal to 10 OR if it is even.

```
if (my_number == 10 || my_number % 2 == 0) { // if this condition is True, then we
enter this if statement  - % is called the modulo operator and returns the
remainder of the two numbers after division
	System.out.println("Specific Message");
}
```

Exercises – Part 1

1. Write a program to check whether two floating-point numbers are equal or not.
2. Write a program to check whether a given number is even or odd.
3. Write a program to find the largest of three numbers.
4. Write a program to, given a grade, declare the equivalent description according to the Common Marking Scheme.

Loops (for)

A *loop* is a functionality that allows you to execute a block of code multiple times. Loops require a condition that is evaluated in every iteration in order to define if the loop should continue or stop. A loop executes instructions repeatedly while this condition is true.

Example: Print all numbers from 0 to 100.

```
for(int i=0; i<=100; i++) {
	System.out.println(i);
}
```

Usually, a loop requires: (i) initialization, i.e., setup variables to start looping – in the previous example, is the declaration of the i variable; (ii) condition, to test if we should execute loop body – in the previous example is the i <= 100; and (iii) update, that changes something each time through – in the previous example is the i++ that increments the value of i.

Methods

A method is a named sequence of instructions defined by the return type, the name, and the parameters. Arguments are supplied, if necessary, when a method is called. The return value is the result that the method computes.

When declaring a method, you must provide a name for the method, a variable for each argument, and a type for the result.

Example: A method that receives two numbers and returns the largest.

```
public static void main(String[] args) {
	int num1 = 10;
	int num2 = 20;

	int max_value = max(num1, num2); // calling the method max passing the two
expected parameters
}

public int max(int a, int b) { // this is a method called max that returns an int
and receives two arguments of type int
	int max;
	if (a > b) {
		max = a;
	}
	else {
		max = b;
	}
	return max;
}
```

In this example, the method *max* expects two parameters of type int. It will then compare the two and return the largest one. When we call the method using *max(num1, num2)*, it actually means that, inside the method, the method parameters *int a* and *int b* actually represent the variables *num1* and *num2*, respectively. In other words, if we print *a* inside the method, we would get the value 10 (i.e., the value of variable *num1*). The same applies to *b* and *num2*.

Arrays

An array collects a sequence of values of the same type. Individual elements in an array are accessed by an integer index (usually *i*), using the notation *values[index]*. An array index must be at least zero and less than the size of the array.

Attention: A bounds error, which occurs if you supply an invalid array index, can cause your program to terminate.

Example: Print all numbers of an array of int.

```
int[] array = new int[3]; // new array of int with size 3
array[0] = 5;  // assign 5 as value of position 0
array[1] = 10;  // assign 10 as value of position 1
array[2] = 15;  // assign 15 as value of position 2

// array - [5, 10 ,15]
for (int i=0; i< array.length; i++) { // array.length returns the length (or size)
of the array
	System.out.println(array[i]);
}
```

In this example, the variable *array* can only store integers, as it was declared as an array of int.

Exercises – Part 2

1. Write a method that checks whether at least one character in a given string is a vowel (a, e, i, o, u) or not. Return true if the string contains a vowel, otherwise return false.
2. Write a program to construct the following pattern, using loops.

```
*

* *

* * *

* * * *

* * * * *

* * * *

* * *

* *

*
```

3. Write a Java program which takes a digit m and generates an array with size m. The element value in the i-th position of the array should be equal 2*i.
4. Write a Java program which takes two digits m (row) and n (column) and generates a two-dimensional array. The element value in the i-th row and j-th column of the array should be i*j (i.e., i=0, j=0 => 0*0; i=5, j=3 => 5*3).